TERM PROJECT 1

CODE:

```python
from LinkedBinaryTree import LinkedBinaryTree as Tree

# Equation 1
tree = Tree()
print("Equation 1: (3 * 5) - ((4 * 5) + (6 - 7))")
tree._add_root('-')
tree._add_left(tree.root(), e: '*')
tree._add_right(tree.root(), e: '+')
tree._add_left(tree.left(tree.root()), e: 3)
tree._add_right(tree.left(tree.root()), e: 5)
tree._add_left(tree.right(tree.root()), e: '*')
tree._add_right(tree.right(tree.root()), e: '-')
tree._add_left(tree.left(tree.right(tree.root())), e: 4)
tree._add_right(tree.left(tree.right(tree.root())), e: 5)
tree._add_left(tree.right(tree.right(tree.root())), e: 6)
tree._add_right(tree.right(tree.right(tree.root())), e: 7)

print("Preorder traversal: ", end=" ")
for i in tree.positions():
    print(i.element(), end=" ")
print()

print("Postorder traversal: ", end=" ")
for i in tree.postorder():
    print(i.element(), end=" ")
print()

print("Inorder traversal: ", end=" ")
for i in tree.inorder():
    print(i.element(), end=" ")
print()
print()

# Equation 2
tree = Tree()
print("Equation 2: ((a + b) * c) - (d - e)")
tree._add_root('-')
tree._add_left(tree.root(), e: '*')
tree._add_right(tree.root(), e: '-')
tree._add_left(tree.left(tree.root()), e: '+')
tree._add_right(tree.left(tree.root()), e: 'c')
tree._add_left(tree.right(tree.root()), e: 'd')
tree._add_right(tree.right(tree.root()), e: 'e')
tree._add_left(tree.left(tree.left(tree.root())), e: 'a')
tree._add_right(tree.left(tree.left(tree.root())), e: 'b')

print("Preorder traversal: ", end=" ")
for i in tree.positions():
    print(i.element(), end=" ")
print()

print("Postorder traversal: ", end=" ")
for i in tree.postorder():
    print(i.element(), end=" ")
print()

print("Inorder traversal: ", end=" ")
for i in tree.inorder():
    print(i.element(), end=" ")
print()
print()

# Equation 3
tree = Tree()
print("Equation 3: ((a ^ b) + (c + d)) + ((e * f) / (g + h))")
tree._add_root('+')
tree._add_left(tree.root(), e: '+')
tree._add_right(tree.root(), e: '/')
tree._add_left(tree.left(tree.root()), e: '^')
tree._add_right(tree.left(tree.root()), e: '+')
tree._add_left(tree.right(tree.root()), e: '*')
tree._add_right(tree.right(tree.root()), e: '+')
tree._add_left(tree.left(tree.left(tree.root())), e: 'a')
tree._add_right(tree.left(tree.left(tree.root())), e: 'b')
tree._add_left(tree.right(tree.left(tree.root())), e: 'c')
tree._add_right(tree.right(tree.left(tree.root())), e: 'd')
tree._add_left(tree.left(tree.right(tree.root())), e: 'e')
tree._add_right(tree.left(tree.right(tree.root())), e: 'f')
```

```python
tree._add_left(tree.right(tree.right(tree.root())), e: 'g')
tree._add_right(tree.right(tree.right(tree.root())), e: 'h')

print("Preorder traversal: ", end=" ")
for i in tree.positions():
    print(i.element(), end=" ")
print()

print("Postorder traversal: ", end=" ")
for i in tree.postorder():
    print(i.element(), end=" ")
print()

print("Inorder traversal: ", end=" ")
for i in tree.inorder():
    print(i.element(), end=" ")
print()
print()

# Equation 4
tree = Tree()
print("Equation 4: (a + b) / (c * (d - (e ^ f)))")
tree._add_root('/')
tree._add_left(tree.root(), e: '+')
tree._add_right(tree.root(), e: '*')
tree._add_left(tree.left(tree.root()), e: 'a')
tree._add_right(tree.left(tree.root()), e: 'b')
tree._add_left(tree.right(tree.root()), e: 'c')
tree._add_right(tree.right(tree.root()), e: '-')
tree._add_left(tree.right(tree.right(tree.root())), e: 'd')
tree._add_right(tree.right(tree.right(tree.root())), e: '^')
tree._add_left(tree.right(tree.right(tree.right(tree.root()))), e: 'e')
tree._add_right(tree.right(tree.right(tree.right(tree.root()))), e: 'f')

print("Preorder traversal: ", end=" ")
for i in tree.positions():
    print(i.element(), end=" ")
print()
```

```python
print("Postorder traversal: ", end=" ")
for i in tree.postorder():
    print(i.element(), end=" ")
print()

print("Inorder traversal: ", end=" ")
for i in tree.inorder():
    print(i.element(), end=" ")
print()
print()

# Equation 5
tree = Tree()
print("Equation 5: ((a - b) + c) * ((d + e) * (f / g))")
tree._add_root('*')
tree._add_left(tree.root(), e: '+')
tree._add_right(tree.root(), e: '*')
tree._add_left(tree.left(tree.root()), e: '-')
tree._add_right(tree.left(tree.root()), e: 'c')
tree._add_left(tree.right(tree.root()), e: '+')
tree._add_right(tree.right(tree.root()), e: '/')
tree._add_left(tree.left(tree.left(tree.root())), e: 'a')
tree._add_right(tree.left(tree.left(tree.root())), e: 'b')
tree._add_left(tree.left(tree.right(tree.root())), e: 'd')
tree._add_right(tree.left(tree.right(tree.root())), e: 'e')
tree._add_left(tree.right(tree.right(tree.root())), e: 'f')
tree._add_right(tree.right(tree.right(tree.root())), e: 'g')

print("Preorder traversal: ", end=" ")
for i in tree.positions():
    print(i.element(), end=" ")
print()

print("Postorder traversal: ", end=" ")
for i in tree.postorder():
    print(i.element(), end=" ")
print()
```

```python
print("Inorder traversal: ", end=" ")
for i in tree.inorder():
    print(i.element(), end=" ")
print()
print()

# Equation 6
tree = Tree()
print("Equation 6: (((5 + 2) * (2 - 1)) / ((2 + 9) + ((7 - 2) - 1)) * 8")
tree._add_root('*')
tree._add_left(tree.root(), e: '/')
tree._add_right(tree.root(), e: '8')
tree._add_left(tree.left(tree.root()), e: '*')
tree._add_right(tree.left(tree.root()), e: '+')
tree._add_left(tree.left(tree.left(tree.root())), e: '+')
tree._add_right(tree.left(tree.left(tree.root())), e: '-')
tree._add_left(tree.right(tree.left(tree.root())), e: '+')
tree._add_right(tree.right(tree.left(tree.root())), e: '-')
tree._add_left(tree.left(tree.left(tree.left(tree.root()))), e: '5')
tree._add_right(tree.left(tree.left(tree.left(tree.root()))), e: '2')
tree._add_left(tree.right(tree.left(tree.left(tree.root()))), e: '2')
tree._add_right(tree.right(tree.left(tree.left(tree.root()))), e: '1')
tree._add_left(tree.left(tree.right(tree.left(tree.root()))), e: '2')
tree._add_right(tree.left(tree.right(tree.left(tree.root()))), e: '9')
tree._add_left(tree.right(tree.right(tree.left(tree.root()))), e: '-')
tree._add_right(tree.right(tree.right(tree.left(tree.root()))), e: '1')
tree._add_left(tree.left(tree.right(tree.right(tree.left(tree.root())))), e: '7')
tree._add_right(tree.left(tree.right(tree.right(tree.left(tree.root())))), e: '2')

print("Preorder traversal: ", end=" ")
for i in tree.positions():
    print(i.element(), end=" ")
print()

print("Postorder traversal: ", end=" ")
for i in tree.postorder():
    print(i.element(), end=" ")
print()
```

```python
print("Inorder traversal: ", end=" ")
for i in tree.inorder():
    print(i.element(), end=" ")
print()
print()
```

OUTPUT:

```
Equation 1: (3 * 5) - ((4 * 5) + (6 - 7))
Preorder traversal:  - * 3 5 + * 4 5 - 6 7
Postorder traversal:  3 5 * 4 5 * 6 7 - + -
Inorder traversal:  3 * 5 - 4 * 5 + 6 - 7

Equation 2: ((a + b) * c) - (d - e)
Preorder traversal:  - * + a b c - d e
Postorder traversal:  a b + c * d e - -
Inorder traversal:  a + b * c - d - e

Equation 3: ((a ^ b) + (c + d)) + ((e * f) / (g + h))
Preorder traversal:  + + ^ a b + c d / * e f + g h
Postorder traversal:  a b ^ c d + + e f * g h + / +
Inorder traversal:  a ^ b + c + d + e * f / g + h

Equation 4: (a + b) / (c * (d - (e ^ f)))
Preorder traversal:  / + a b * c - d ^ e f
Postorder traversal:  a b + c d e f ^ - * /
Inorder traversal:  a + b / c * d - e ^ f

Equation 5: ((a - b) + c) * ((d + e) * (f / g))
Preorder traversal:  * + - a b c * + d e / f g
Postorder traversal:  a b - c + d e + f g / * *
Inorder traversal:  a - b + c * d + e * f / g

Equation 6: (((5 + 2) * (2 - 1)) / ((2 + 9) + ((7 - 2) - 1))) * 8
Preorder traversal:  * / * + 5 2 - 2 1 + + 2 9 - - 7 2 1 8
Postorder traversal:  5 2 + 2 1 - * 2 9 + 7 2 - 1 - + / 8 *
Inorder traversal:  5 + 2 * 2 - 1 / 2 + 9 + 7 - 2 - 1 * 8
```